

MemoSim

Memory allocation simulator using Segmentation with First-Fit and Best-Fit Algorithms

CSE 335s – Operating Systems Memory Management Assignment

Submitted by: Youssef Yacoub Radi

Student ID: 2200649

Date: May 10, 2026

Drive Link (Source codes + .exe file):

[https://drive.google.com/drive/folders/1VdybcmtPoaC61DJ0hyXY2KhOAYB-HNfc?usp=drive link](https://drive.google.com/drive/folders/1VdybcmtPoaC61DJ0hyXY2KhOAYB-HNfc?usp=drive_link)

Contents

1. Introduction	3
2. Theoretical Background.....	4
2.1 Segmentation	4
2.2 Dynamic Storage-Allocation Problem.....	4
2.3 External Fragmentation and Coalescing.....	4
3. System Design and Architecture.....	5
3.1 Directory Structure.....	5
3.2 Controller Layer.....	5
3.3 UI Layer	5
4. Data Structures	5
5. Implementation Details.....	6
5.1 Atomic Allocation.....	6
5.2 Hole Management and Coalescing.....	6
5.3 First-Fit vs Best-Fit Logic.....	6
6. User Manual	7
7. Test Cases and Results.....	8
7.1 First-Fit Execution	8
7.2 Best-Fit Execution.....	11
8. Discussion on the test provided.....	17
9. Conclusion	17

1. Introduction

Memory management is one of the most fundamental responsibilities of an operating system. In this project, we developed a graphical simulation tool that demonstrates how an OS allocates and deallocates memory using segmentation. Unlike paging, which divides memory into fixed-size blocks, segmentation treats a program as a collection of logical units—such as code, data, and stack—each with its own size. This approach aligns more closely with how programmers view their programs.

Our simulator supports two well-known dynamic storage-allocation strategies: First-Fit and Best-Fit. The goal was not only to implement the algorithms correctly but also to visualize the internal state of memory after every operation. By doing so, we can clearly observe phenomena like external fragmentation and hole coalescing, which are often hard to grasp from textbooks alone.

The application was built in Python using Tkinter for the GUI. We chose Python because it allows rapid prototyping, and Tkinter is bundled with the standard library, making deployment straightforward. The project is structured into two main packages: Controller (business logic) and UI (presentation layer), following a simple MVC-inspired pattern.

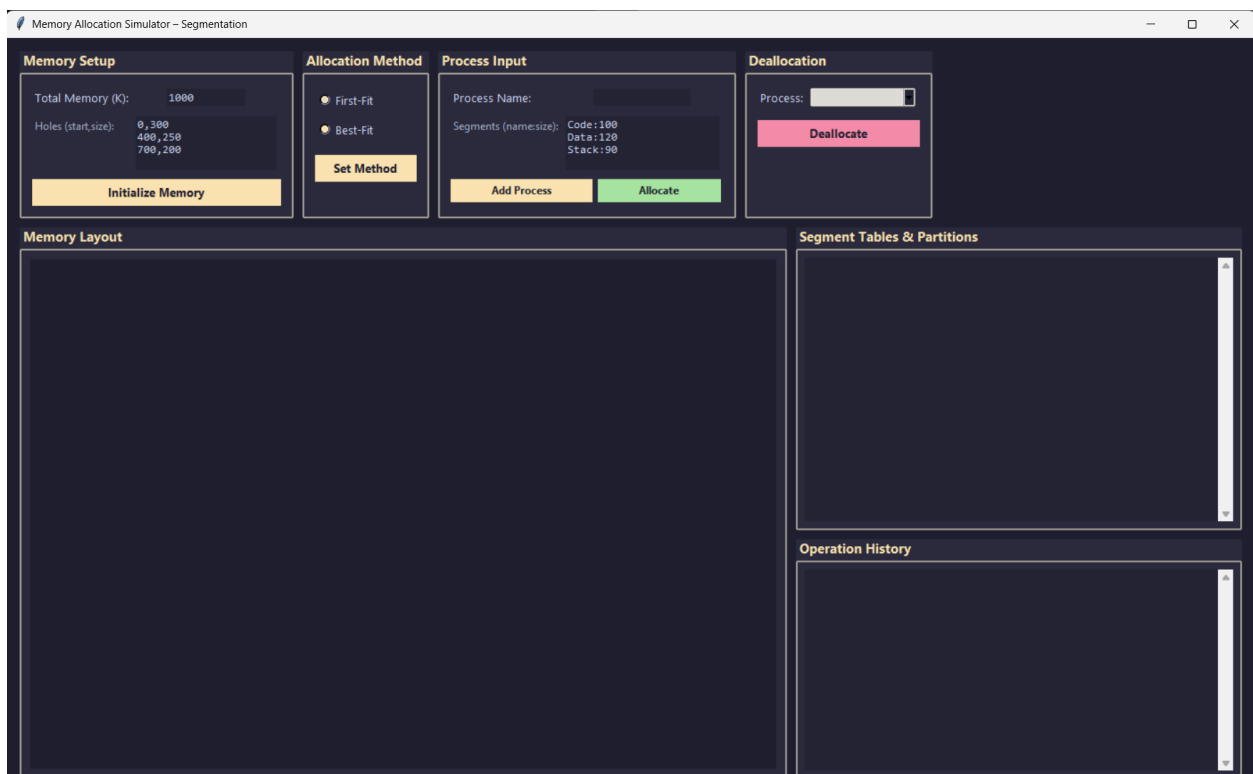


Figure 1: Main Application Window

2. Theoretical Background

2.1 Segmentation

Segmentation is a memory-management scheme that supports the user's view of memory. A program is naturally divided into segments: the main program, procedures, functions, local and global variables, symbol tables, and stacks. Each segment is a logical unit with a name and a length. The logical address generated by the CPU is a pair <segment-number, offset>.

The operating system maintains a segment table for each process. Each entry in this table contains two critical values:

- Base – the starting physical address where the segment resides in memory.
- Limit – the length of the segment, used for bounds checking.

When the CPU generates a logical address, the segmentation hardware uses the segment number to index the segment table, verifies that the offset is less than the limit, and then adds the base address to the offset to produce the physical address. If the offset exceeds the limit, a trap is raised, preventing illegal memory access.

2.2 Dynamic Storage-Allocation Problem

Because segments vary in length, memory allocation becomes a dynamic storage-allocation problem. The OS must keep track of which parts of memory are currently in use and which are free. Free memory regions are called holes, and they are scattered throughout memory. When a process arrives, the allocator must find a hole large enough to accommodate the requested segment.

We implemented two strategies to solve this problem:

1. First-Fit: Allocate the first hole that is big enough. The search starts from the beginning of the hole list and stops at the first match.
2. Best-Fit: Allocate the smallest hole that is big enough. This requires searching the entire list (unless it is sorted by size) but leaves the smallest possible leftover fragment.

In practice, First-Fit is faster because it minimizes search time. Best-Fit tends to produce better memory utilization in the short term, but it can generate many tiny, unusable holes—a phenomenon known as external fragmentation.

2.3 External Fragmentation and Coalescing

External fragmentation occurs when the total free memory is sufficient to satisfy a request, but no single contiguous block is large enough. This is a common issue in segmentation-based systems. To mitigate it, the OS performs coalescing: when a process is deallocated, its segments become holes, and the system checks whether these new holes are adjacent to existing ones. If so, they are merged into a single larger hole. Our simulator performs this merging automatically after every deallocation.

3. System Design and Architecture

We designed the project with simplicity and separation of concerns in mind. The architecture is loosely based on the Model-View-Controller (MVC) pattern, though we simplified it into two layers since the scope is a single-user desktop application.

3.1 Directory Structure

```
MemoSim/
├── main.py                # Entry point
├── Controller/
│   ├── __init__.py
│   ├── models.py         # Data classes
│   └── memory_manager.py  # Core allocation logic
└── UI/
    ├── __init__.py
    └── main_app.py        # Tkinter interface
```

3.2 Controller Layer

The Controller package encapsulates all the business logic. It does not depend on Tkinter or any GUI framework, which means it could be reused in a web or command-line version of the tool without modification.

- `models.py` defines four dataclasses: `Segment`, `Process`, `Hole`, and `AllocatedBlock`. These are lightweight containers with no behavior.
- `memory_manager.py` contains the `MemoryManager` class, which is the heart of the simulator. It handles hole management, allocation, deallocation, and coalescing.

3.3 UI Layer

The UI package is responsible for rendering the interface and translating user actions into method calls on the `MemoryManager`. We used a custom dark theme with an amber accent color to reduce eye strain and make the memory layout visually distinct. The layout is horizontal: control panels sit at the top, while the visualization area occupies the bottom half of the window.

4. Data Structures

The following tables summarize the key data structures used in the simulator.

Field	Type	Description
name	str	Logical name of the segment (e.g., Code, Data, Stack).
size	int	Size of the segment in kilobytes.
base	int	Physical starting address: -1 if not allocated.

Table 1: Segment dataclass fields

Attribute	Type	Purpose
total_size	int	Total physical memory size in KB.
holes	List[Hole]	Free memory blocks, kept sorted by start address.
allocated	List[AllocatedBlock]	Currently occupied memory regions.
processes	Dict[str, Process]	Map of process names to their objects.
history	List[str]	Log of all operations for display.
allocation_method	str	Either 'first_fit' or 'best_fit'.

Table 2: MemoryManager internal state

5. Implementation Details

5.1 Atomic Allocation

One of the most important design decisions we made was to implement atomic allocation. A process is allocated only if all of its segments can find suitable holes. If even one segment fails, the entire process is rejected, and no partial allocation occurs. This prevents the system from entering an inconsistent state where a process has some segments in memory and others missing.

To achieve this, we simulate the allocation on a temporary copy of the hole list. Only after every segment has found a valid home do we commit the changes to the real hole list and create the AllocatedBlock entries. This pattern is similar to database transactions and ensures correctness.

5.2 Hole Management and Coalescing

Holes are always kept sorted by starting address. After an allocation, the chosen hole is either removed entirely (exact fit) or shrunk from the front. During deallocation, we create new holes from the freed segments and then run a merge pass. The merge pass scans the sorted list and combines any two adjacent holes into one. This keeps the hole list compact and maximizes the chance that future large requests will succeed.

5.3 First-Fit vs Best-Fit Logic

The `_find_hole()` method encapsulates the difference between the two algorithms. For First-Fit, we simply iterate through the holes in address order and return the index of the first hole whose size is greater than or equal to the requested size. For Best-Fit, we iterate through the entire list, tracking the smallest sufficient hole. If no hole is large enough, we return `None`, and the caller raises a failure message.

```

def _find_hole(self, holes: List[Hole], size: int) -> Optional[int]:
    if self.allocation_method == "first_fit":
        for i, hole in enumerate(holes):
            if hole.size >= size:
                return i
        return None
    else: # best_fit
        best_idx = None
        best_size = float('inf')
        for i, hole in enumerate(holes):
            if hole.size >= size and hole.size < best_size:
                best_size = hole.size
                best_idx = i
        return best_idx

```

```

def allocate_process(self, process: Process) -> bool:
    if process.name in self.processes and self.processes[process.name].is_allocated:
        self.history.append(f"Process {process.name} is already allocated.")
        return False

    # Simulation: check if all segments can fit before committing
    temp_holes = [Hole(h.start, h.size) for h in self.holes]
    planned: List[Tuple[int, int, Segment]] = [] # (start, size, segment)

    for seg in process.segments:
        idx = self._find_hole(temp_holes, seg.size)
        if idx is None:
            self.history.append(
                f"Process {process.name} does not fit "
                f"(segment '{seg.name}' of size {seg.size}K cannot be allocated).")
            return False

        hole = temp_holes[idx]
        planned.append((hole.start, seg.size, seg))

        # Update temp hole
        if hole.size == seg.size:
            temp_holes.pop(idx)
        else:
            hole.start += seg.size
            hole.size -= seg.size
            # Keep sorted by start address for correct first-fit behavior
            temp_holes.sort(key=lambda h: h.start)

    # Commit allocations
    for start, size, seg in planned:
        seg.base = start
        self.allocated.append(
            AllocatedBlock(start, size, process.name, seg.name)
        )
        self._consume_hole(start, size)

    process.is_allocated = True
    self.processes[process.name] = process
    self.history.append(
        f"Allocated process {process.name}: " + ", ".join(
            f"{s.name}={s.size}K@{s.base}" for s in process.segments
        )
    )
    return True

```

Figure 2: Code Snippet – Allocation Logic

6. User Manual

Follow these steps to use the simulator:

3. Launch the application by running main.py or double-clicking the executable (MmemoryAllocator.exe).
4. In the Memory Setup panel, enter the total memory size (default is 1000K) and define the initial holes as comma-separated start, size pairs.
5. Click Initialize Memory to create the memory model.
6. Select either First-Fit or Best-Fit in the Allocation Method panel and click Set Method.
7. In the Process Input panel, type a process name and its segments in the format: **'name: size'** (one per line).
Click Add Process to store it.
8. With the process name still in the field, click Allocate. The canvas and tables will update immediately.
9. To remove a process, select it from the Deallocation dropdown and click Deallocate. The freed space will merge with neighboring holes.

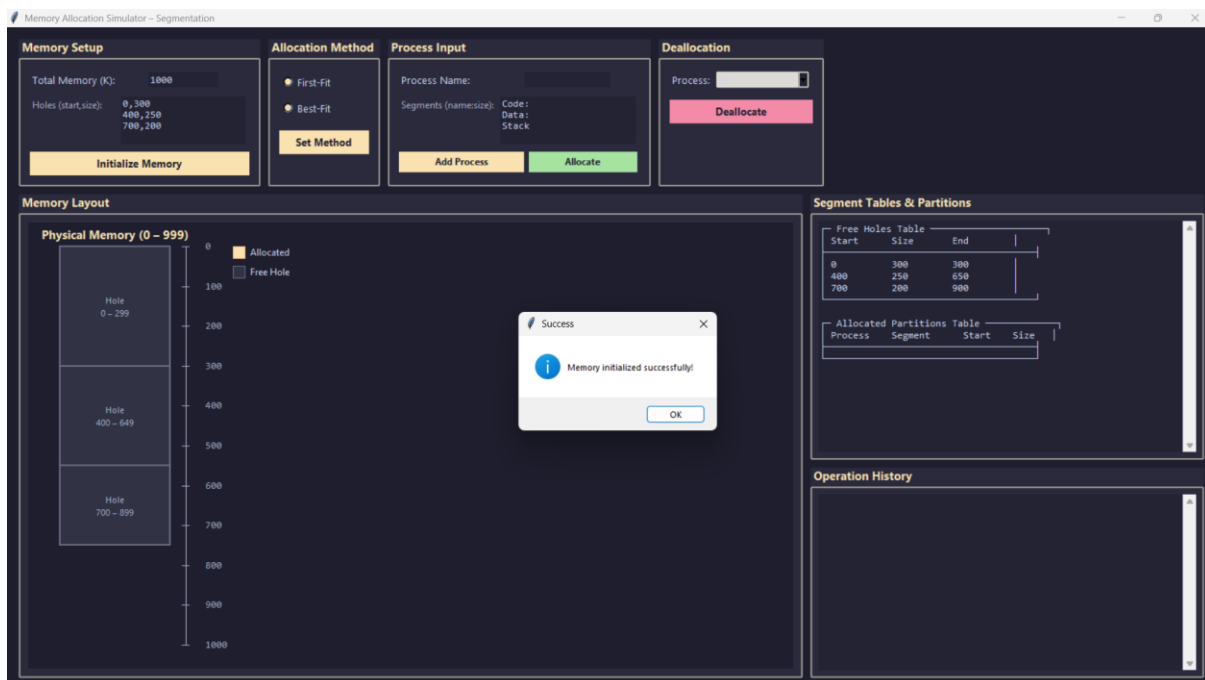


Figure 3: Memory Setup and Initialization

7. Test Cases and Results

This section documents the test scenario provided earlier. We ran the same sequence twice—once with First-Fit and once with Best-Fit—to highlight how the choice of algorithm affects memory layout and fragmentation.

Initial Setup: Total Memory = 1000K; Holes: (0,300), (400,250), (700,200).

7.1 First-Fit Execution

The Test case is traced where the table entry stores the test case step, followed by the screenshot result of the step

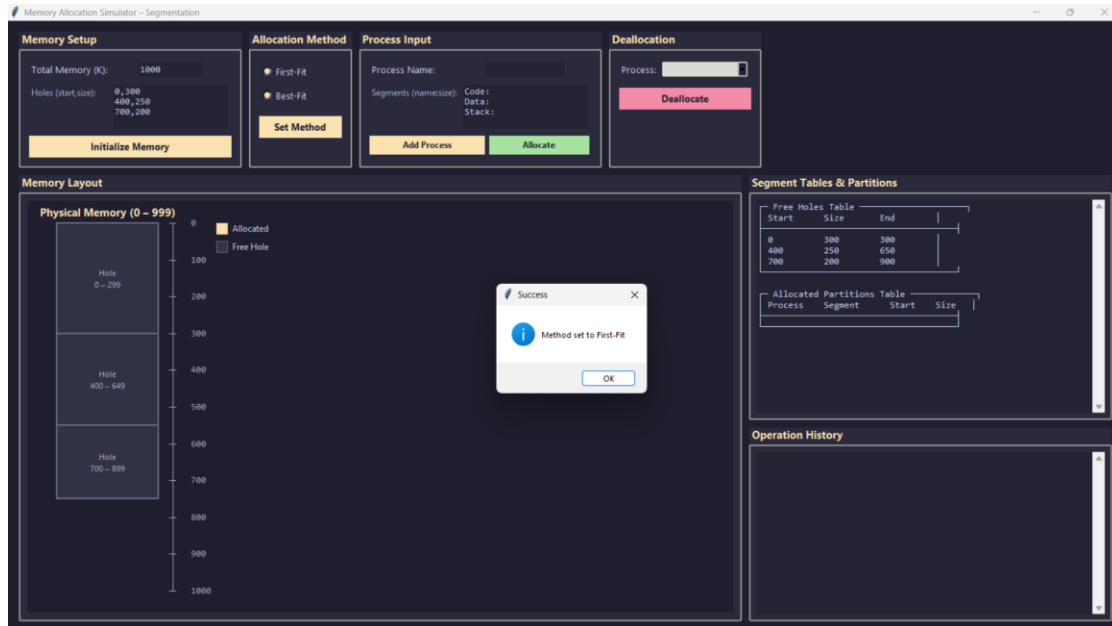


Figure 4: Choosing the First-fit algorithm via the Allocation Method box in the UI

Step	Operation	Request
Allocate P1	Code=100, Data=120, Stack=90	Success. Code→0, Data→100, Stack→400.
Allocate P2	Code=200, Data=40	Success.

Code→700, Data→220.

Memory Allocation Simulator – Segmentation

Memory Setup

Total Memory (K): 1000
Holes (start,size): 0,300
400,250
700,200
Initialize Memory

Allocation Method

First-Fit
Best-Fit
Set Method

Process Input

Process Name: p2
Segments (name,size): Code:200
Data:40
Add Process Allocate

Deallocation

Process: p2
Deallocate

Memory Layout

Physical Memory (0 – 999)
0 100 200 300 400 500 600 700 800 900 1000
p1:Code 0 – 99
p1:Data 100 – 219
p2:Data 220 – 259
p1:Stack 260 – 299
p1:Stack 400 – 489
Hole 490 – 649
p2:Code 700 – 899
Allocated
Free Hole

Segment Tables & Partitions

Process p1 Segment Table

Segment	Base	Limit
Code	0	100
Data	100	120
Stack	400	90

Process p2 Segment Table

Segment	Base	Limit
Code	700	200
Data	220	40
Stack	260	0

Free Holes Table

Operation History

Allocated process p1: Code=100K@0, Data=120K@100, Stack=90K@400
Process p2 does not exist.
Allocated process p2: Code=200K@700, Data=40K@220, Stack=0K@260

Allocate P3

Code=120, Data=50

FAIL.
After placing Code@490, no hole ≥50 remains for Data.

Memory Allocation Simulator – Segmentation

Memory Setup

Total Memory (K): 1000
Holes (start,size): 0,300
400,250
700,200
Initialize Memory

Allocation Method

First-Fit
Best-Fit
Set Method

Process Input

Process Name: p3
Segments (name,size): Code:120
Data:50
Add Process Allocate

Deallocation

Process: p3
Deallocate

Memory Layout

Physical Memory (0 – 999)
0 100 200 300 400 500 600 700 800 900 1000
p1:Code 0 – 99
p1:Data 100 – 219
p2:Data 220 – 259
p1:Stack 260 – 299
p1:Stack 400 – 489
Hole 490 – 649
p2:Code 700 – 899
Allocated
Free Hole

Segment Tables & Partitions

Process p1 Segment Table

Segment	Base	Limit
Code	0	100
Data	100	120
Stack	400	90

Process p2 Segment Table

Segment	Base	Limit
Code	700	200
Data	220	40
Stack	260	0

Free Holes Table

Operation History

Allocated process p1: Code=100K@0, Data=120K@100, Stack=90K@400
Process p2 does not exist.
Allocated process p2: Code=200K@700, Data=40K@220, Stack=0K@260
Process p3 does not fit (segment 'Data' of size 50K cannot be allocated).

Deallocate P1

—

Success.
Holes merged: 0–219 (220K) and 400–649 (250K).

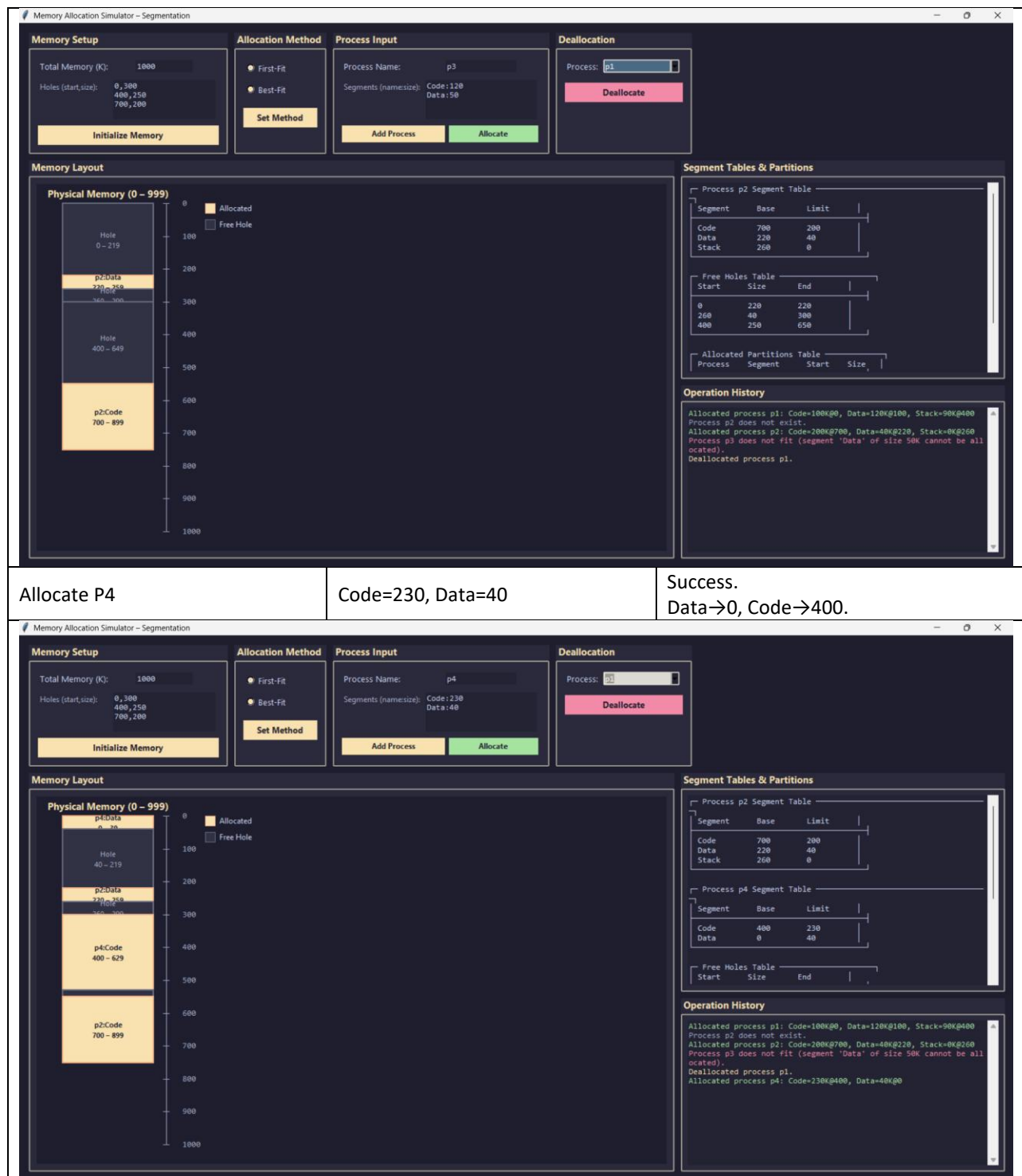


Table 3: First-Fit test execution trace

7.2 Best-Fit Execution

The Test case is traced where the table entry stores the test case step, followed by the screenshot result of the step

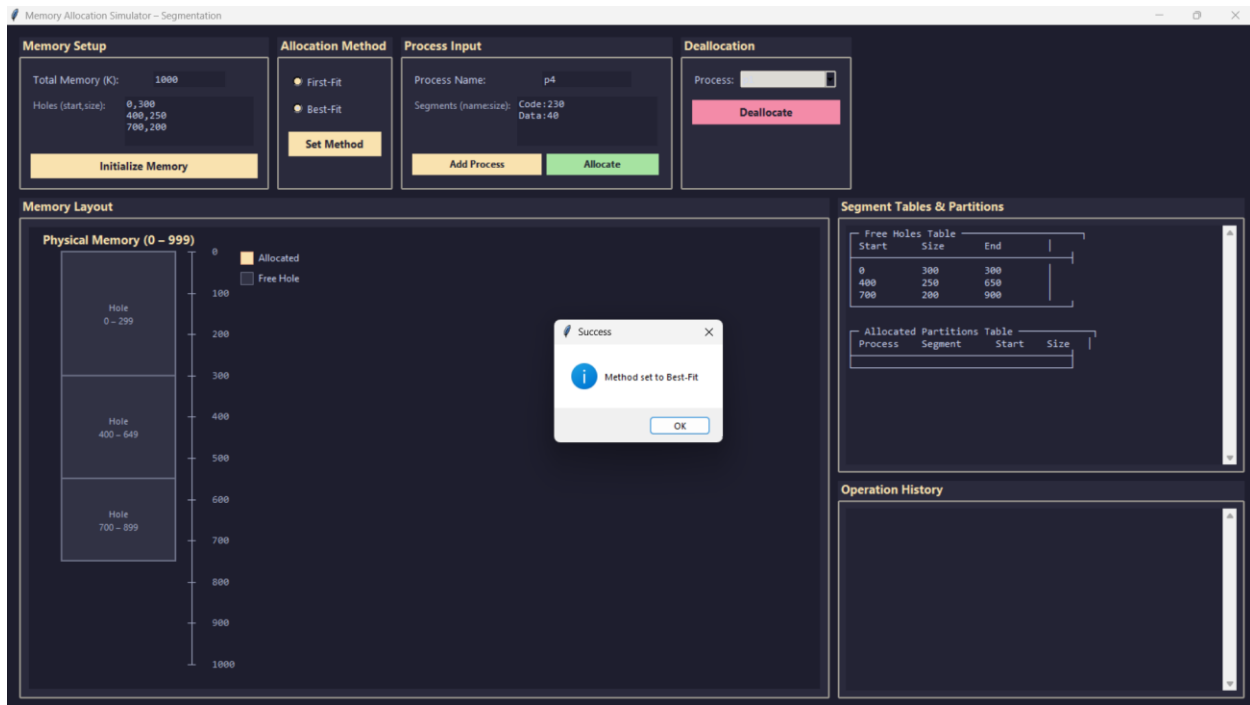
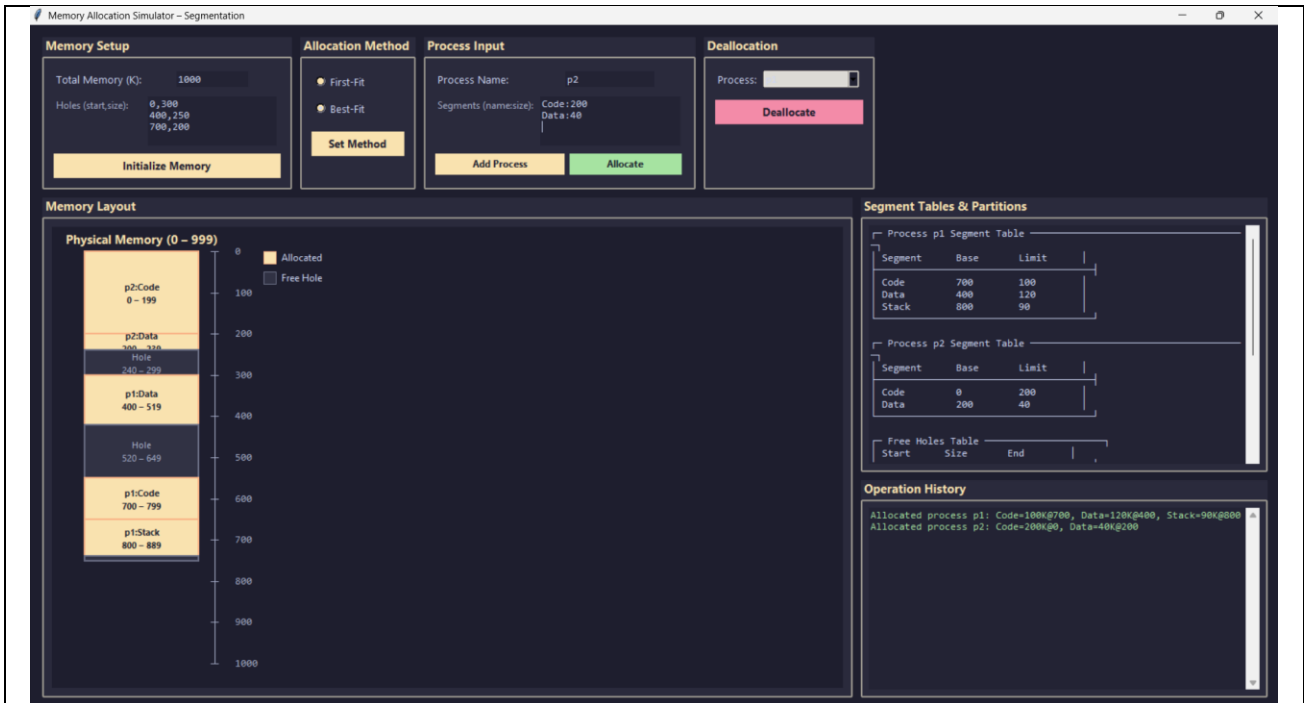


Figure 5: Choosing the Best-first algorithm via the Allocation Method box in the UI

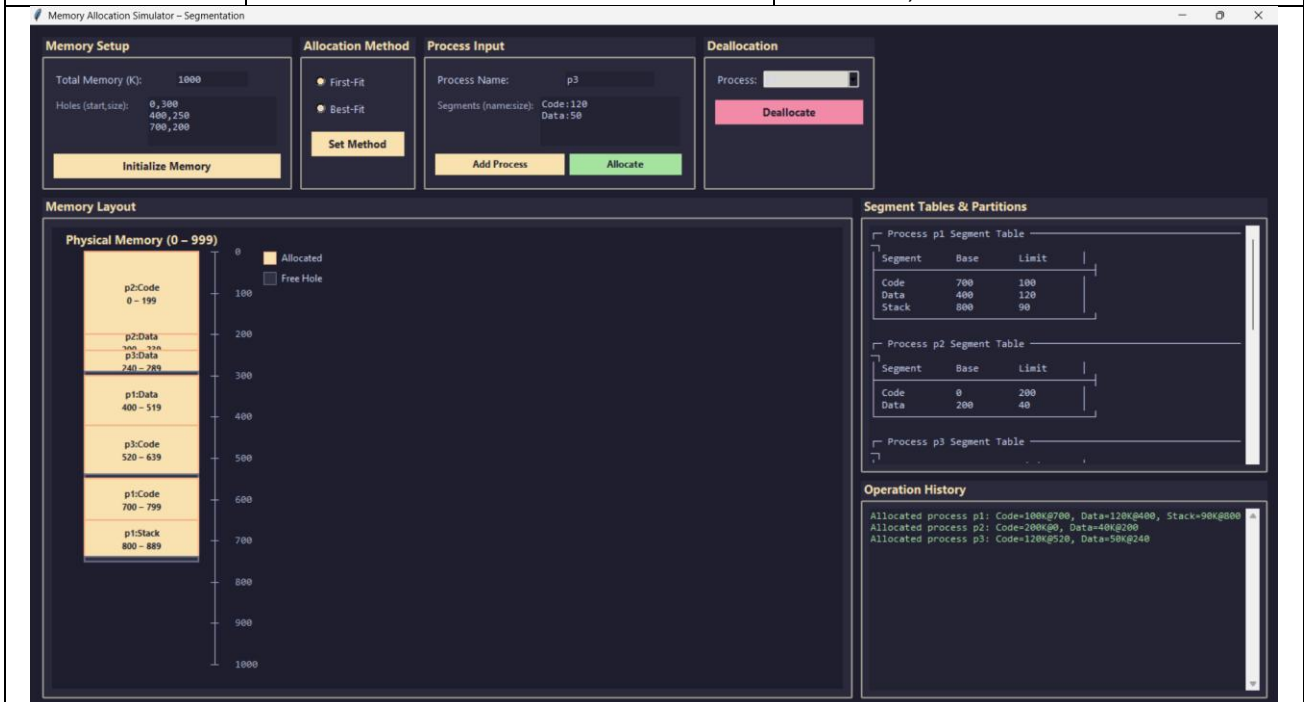
Step	Operation	Request
Allocate P1	Code=100, Data=120, Stack=90	Success. Code→700, Data→400, Stack→800.
The screenshot shows the 'Memory Allocation Simulator - Segmentation' interface after allocating process P1. The 'Process Input' box shows 'p1' with segments: Code:100, Data:120, Stack:90. The 'Memory Layout' section displays a vertical bar representing physical memory from 0 to 999, with holes: 0-299, 400-519 (p1Data), 520-649, 700-799 (p1Code), and 800-899 (p1Stack). The 'Segment Tables & Partitions' section shows the 'Process p1 Segment Table' and 'Free Holes Table'. The 'Operation History' section shows 'Allocated process p1: Code=100@700, Data=120@400, Stack=90@800'.		
Allocate P2	Code=200, Data=40	Success. Code→0, Data→200.



Allocate P3

Code=120, Data=50

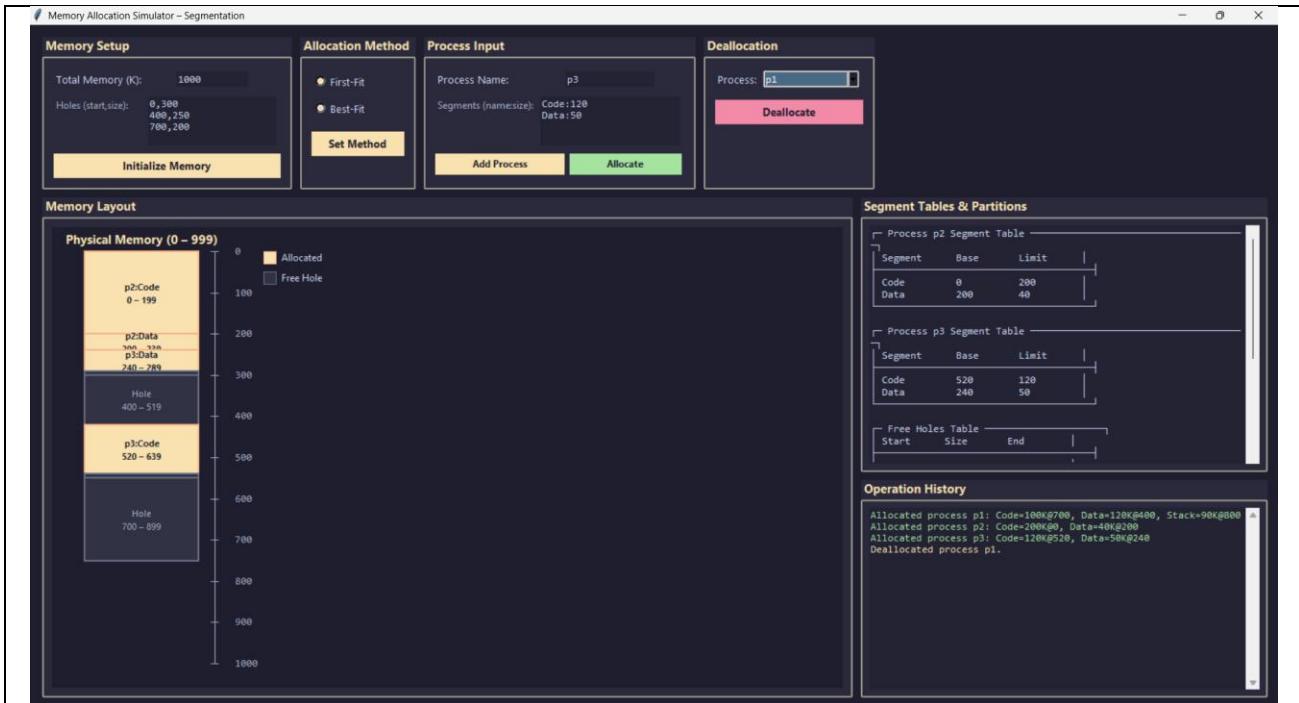
Success.
Code→520, Data→240.



Deallocate P1

—

Success.
Holes created at 400–519 and 700–899.



Allocate P4

Code=230, Data=40

FAIL. Largest hole is 200K; 230K cannot fit.

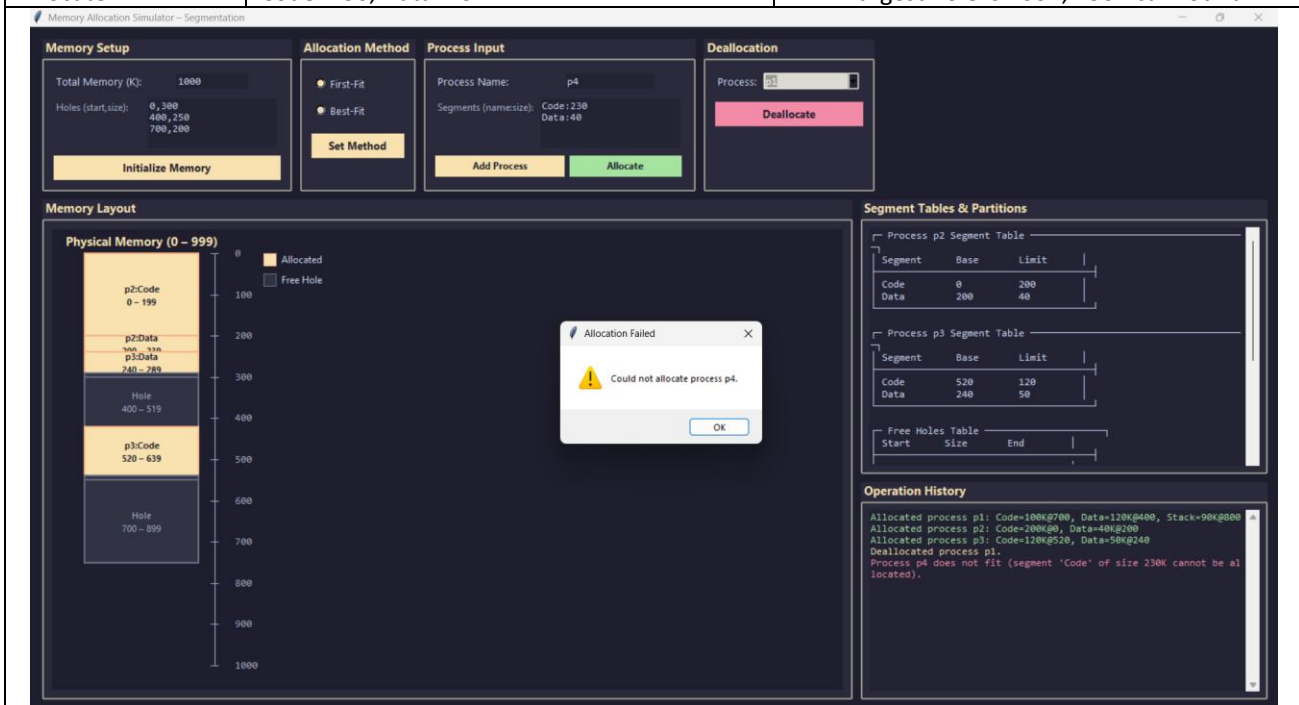


Table 4: Best-Fit test execution trace

7.3 Additional Verification Tests

Beyond the instructor test cases, we ran several extra scenarios to verify edge cases such as exact-fit allocation, atomic rejection, progressive hole merging, and Best-Fit fragmentation behavior. All tests passed successfully.

- Exact Fit & Coalescing: Allocated exact hole sizes, deallocated, and confirmed holes return to original positions without false merging.

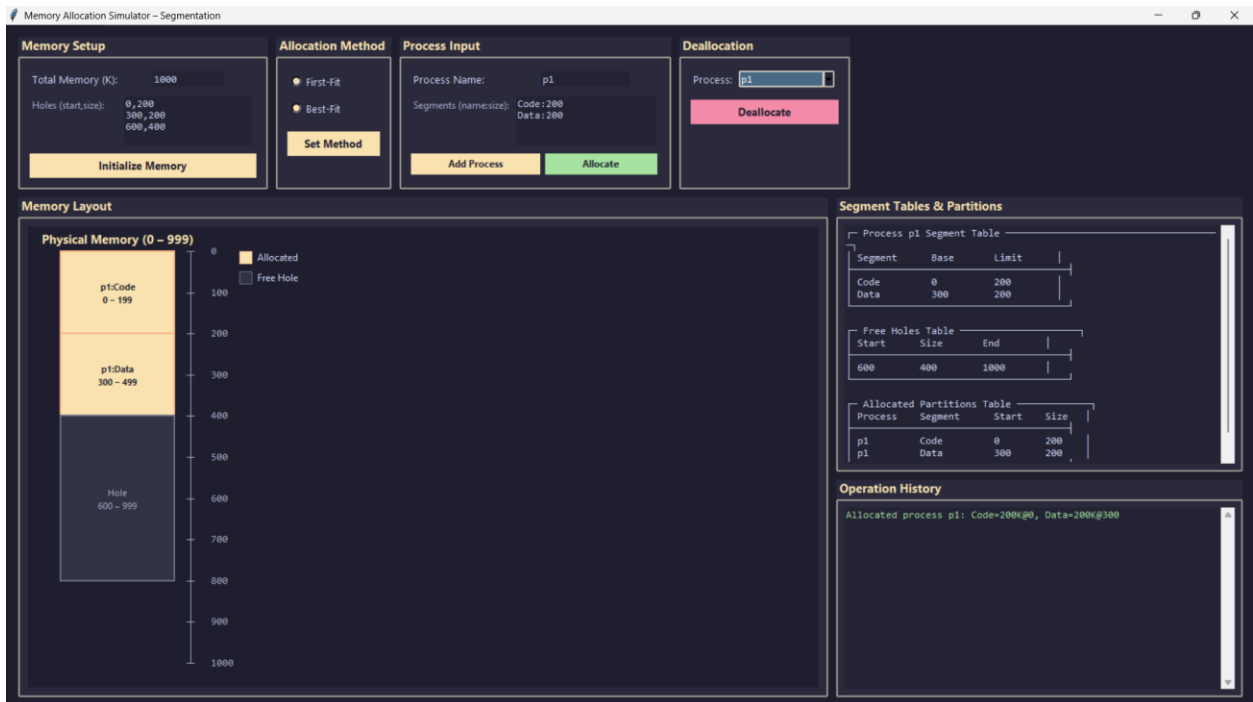


Figure 6.1: Extra Test – Exact fit Allocation

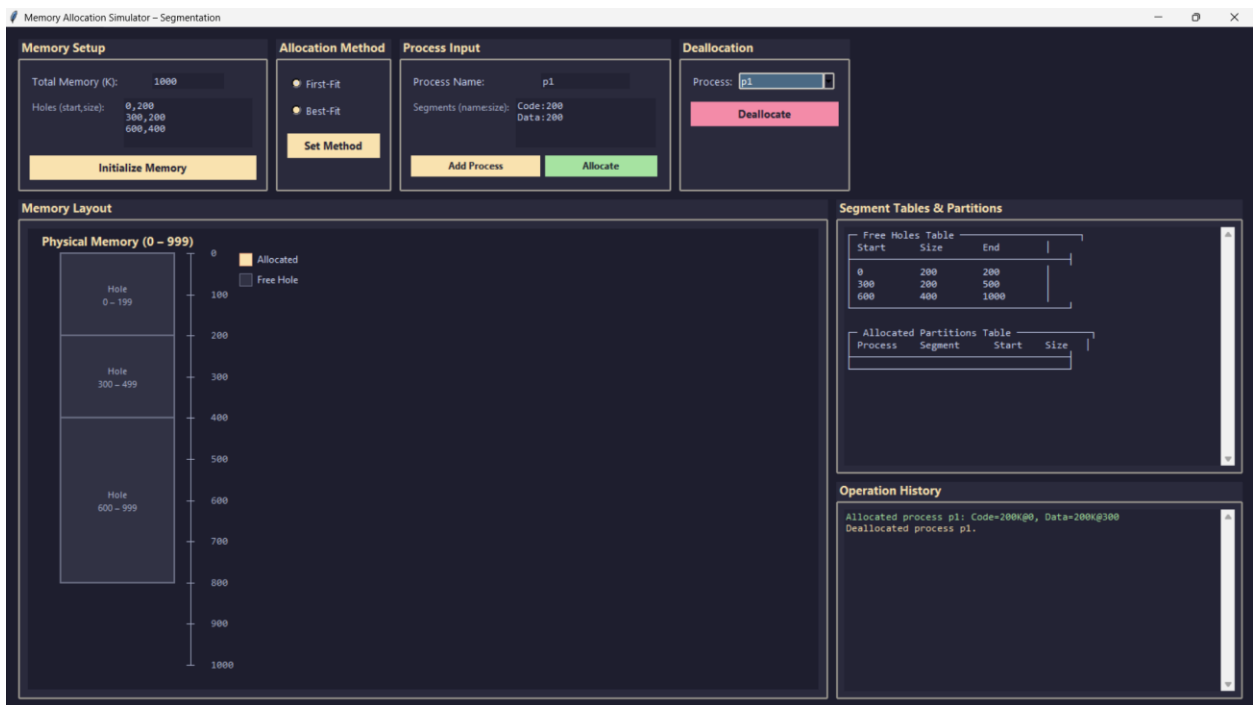


Figure 6.2: Extra Test – Exact fit Deallocation

- Atomic Allocation: Attempted to allocate 3 segments into 2 holes; the process was correctly rejected with zero partial blocks.

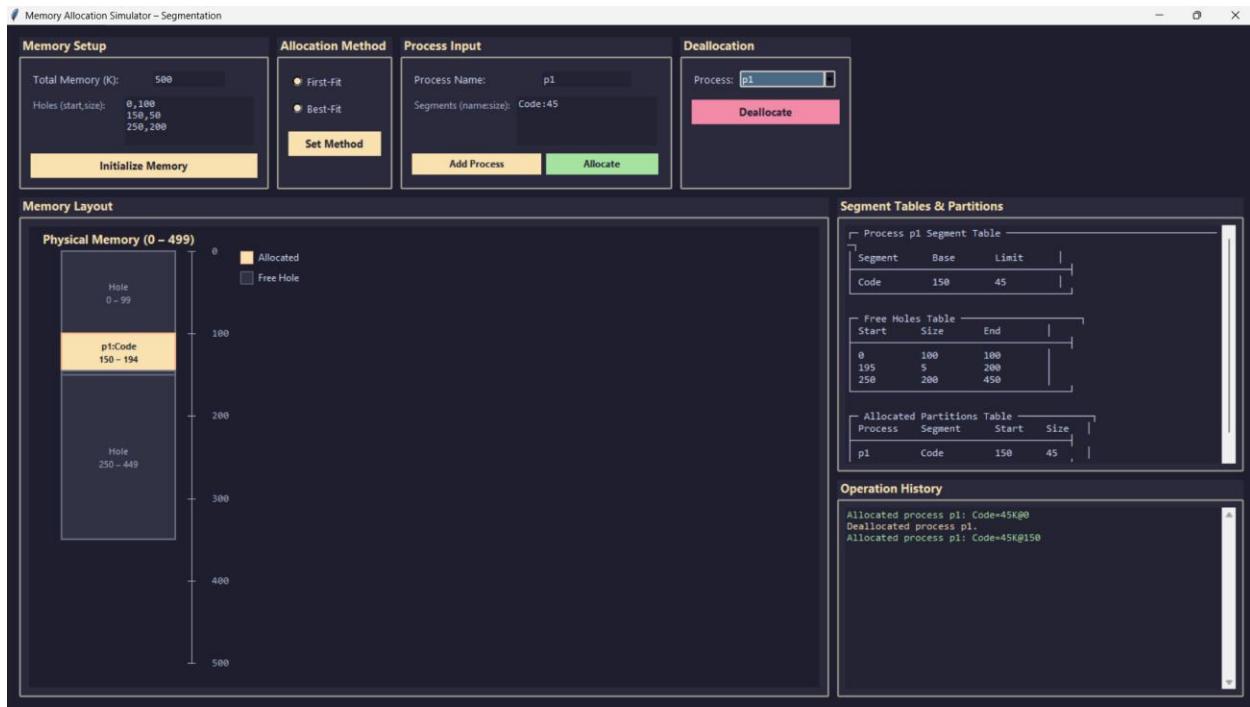


Figure 6.3: Extra Test – Best-fit allocation

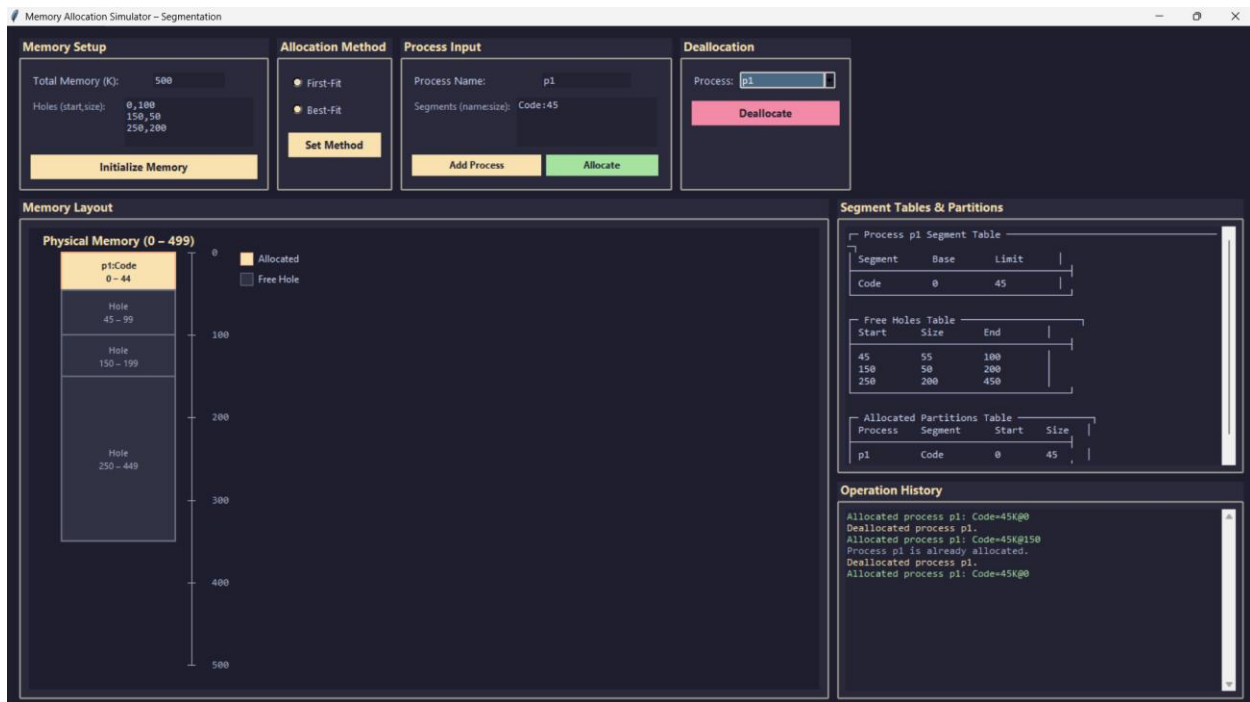


Figure 6.4: Extra Test – First-fit allocation

8. Discussion on the test provided

The test results clearly illustrate the trade-offs between First-Fit and Best-Fit.

First-Fit tends to cluster allocations at the low end of memory, leaving larger holes at higher addresses. This explains why P3 failed under First-Fit: the algorithm consumed the 300K hole at address 0 for P1's Code and Data, leaving only 80K there, then placed P1's Stack at 400, fragmenting the middle hole. By the time P3 arrived, the remaining holes were too small to hold both its segments.

Best-Fit, on the other hand, distributed the segments more evenly across the available holes. It placed P1's segments in the 250K and 200K holes, preserving the large 300K hole for P2. This allowed P3 to squeeze into the remaining fragments. However, Best-Fit's precision came at a cost: after deallocating P1, the freed spaces were 120K and 200K, neither of which could accommodate P4's 230K Code segment. The tiny 10K and 40K fragments left behind by earlier allocations were essentially wasted.

These observations align with the theoretical analysis from our textbook:

Neither algorithm is universally superior. First-Fit is generally faster and produces larger leftover holes on average, while Best-Fit minimizes per-allocation waste but increases external fragmentation over time.

For systems where large segments are common, First-Fit may provide better long-term utilization because it avoids the proliferation of tiny holes.

9. Conclusion

In this project, we successfully designed and implemented a memory allocation simulator that demonstrates segmentation-based memory management. The tool supports both First-Fit and Best-Fit algorithms, enforces atomic allocation to prevent partial process loading, and automatically coalesces adjacent holes upon deallocation. The graphical interface shows the memory layout, segment tables, and hole tables after every operation.